

e ops chef repo.

This is not usually available to team members. So after opening an MR, get it reviewed and merged by someone with access - mostly someone from Infrastructure team.

Changing version of GitLab CI runner

To be performed by any Distribution team member:

1. Create a new merge request on the chef repo that updates the runner version
1. Ensure that CI passes, and the MR is reviewed by another team member
1. Merge the change into the chef repo
1. Login to the node and run,

```
sh
sudo /root/runnerupgrade.sh
```

to perform the upgrade. This will stop the chef-client service, stop the runner and cleanup the machines, run the chef-client to fetch the new version and finally, start GitLab Runner again.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/maintenance/dev-gitlab-org.md

Common links

Distribution Team Handbook
Distribution Team Infrastructure and Maintenance

dev.gitlab.org

This is an internal GitLab instance running GitLab CE. The omnibus-gitlab package on this server is a stock package with required configuration to keep it operational. Regular omnibus-gitlab commands can be used on this node.

Automated tasks

1. Nightly builds: Every day at 1:30 UTC, a nightly build gets triggered on dev.gitlab.org. The cron trigger times are currently defined at the scheduled pipeline page on dev.gitlab.org.
1. Deployments: Every day at 7:20 UTC, the nightly CE packages gets automatically deployed on dev.gitlab.org. Any errors in the install process will be logged in Sentry.

Slack notifications will appear in dev-gitlab. The cron task is currently defined in role file.

Maintenance tasks

It is Distribution team's responsibility to make sure that the GitLab instance on this server is operational.

Requirements:

Access to the node

Depending on whether the task requires permanent changes to `/etc/gitlab/gitlab.rb`, access to the Chef repo.

If you do not have access to this repository, make sure you create an issue in Infrastructure issue tracker and label it access request.

Manually upgrading/downgrading packages

In case of an issue with the latest deploy, we might need to revert the installation to a previous nightly version and lock the deployment until the fixes are ready. This is done to ensure stability of dev.gitlab.org for others using the instance.

1. To start, create an issue in a team-tasks [issue-tracker] detailing that you will be downgrading installed version, and adding links to related issues. Assign the issue to yourself.

1. Next, make an announcement in announcements slack channel before downgrading the package:

text

I will be manually downgrading package on dev.gitlab.org to <version> as latest nightly is not working as expected. <link to issue>

1. Stop sidekiq and unicorn to be sure that data doesn't get altered during the upgrade.

bash

```
sudo gitlab-ctl stop sidekiq
sudo gitlab-ctl stop unicorn
```

1. Find the previous working version of the package and downgrade to this version:

bash

```
sudo apt-get install gitlab-ce=<version to be installed>
```

For example, if the version is 10.4.0+nightly.75436.44501791-0, you would run:

```
bash
sudo apt-get install gitlab-ce=10.4.0+nightly.75436.44501791-0
```

This will automatically run reconfigure and apply the necessary changes.

1. Once the reconfigure is done, confirm all the services are up and running.

```
bash
sudo gitlab-ctl status
```

1. Confirm the correct version is deployed by visiting <https://dev.gitlab.org/help>.

1. Create a package hold to prevent auto-upgrade:

```
bash
sudo apt-mark hold gitlab-ce
```

and verify the hold is in place.

```
bash
sudo apt-mark showhold
```

1. Back in the announcements channel, leave a message that the downgrade is completed:

```
text
Downgrade completed. The package has also been put on hold to prevent automatic upgrades.
<link to issue>
```

Once the issue has been resolved, unhold the package and upgrade to the latest version.

1. Start by announcing this in announcements channel

```
text
I will be removing the package hold and manually upgrading package on dev.gitlab.org to the
latest nightly. <link to issue>
```

1. Next, unhold the package:

```
bash
sudo apt-mark unhold gitlab-ce
```

1. Continue with upgrading:

```
bash
sudo apt-get update
sudo apt-get install gitlab-ce
```

1. Once the upgrade is completed, verify that the latest version is installed by visiting <https://dev.gitlab.org/help>.

1. Finally, leave a note at the announcements channel

```
console
Upgrade completed. dev.gitlab.org now runs <version>.
```

Changing GitLab configuration

If, for some reason, you need to apply a change to `/etc/gitlab/gitlab.rb`, this change needs to be introduced in the `dev-gitlab-org` role.

If you do not have access to this repository, but you need to do a hot-patch or configuration testing, the following steps can be performed:

1. Stop chef-client on this node:

```
console
sudo service chef-client stop
```

1. Make the necessary change to get the instance running again. If that requires change in `gitlab.rb` file, change it manually and run `reconfigure`.

1. Reach out to Production team to get help on getting your `gitlab.rb` configuration change committed to the Chef server.

1. After this has been applied, start the chef-client on the node: ``sudo service chef-client start``

1. Make sure that any change you did is noted in an issue! It is your responsibility to revert the change on this node once the fix is in place in the package!

SECTION: engineering/infrastructure-platforms/gitlab-delivery/build/maintenance/renovate.md

Common links

Renovate
Renovate docs

Introduction

We use Renovate to monitor external repositories, and open a merge request to the appropriate project when an upgrade is released.

The projects currently using this are

omnibus-gitlab
 Open Merge Requests
CNG images
 Open Merge Requests
charts/gitlab
 Open Merge Requests
cloud-native/gitlab-operator
 Open Merge Requests

All dependencies for a repository are defined in the Renovate bot repository.

Handling Merge Requests

When a merge request for the project is opened, Renovate will assign a reviewer using reviewer roulette.

It is the responsibility of the entire Distribution team to ensure the merge requests are handled in a timely fashion. Team members should assign available merge requests to themselves, removing the other assignees, when they are going to work on them. The team member needs to determine the appropriate milestone to target for the upgrade, and verify the new software version works as expected. If everything looks good, a changelog entry should be added, and the merge request assigned to a maintainer.

omnibus-gitlab

The pipeline for the merge requests should run a triggered pipeline, which will build a package, and run gitlab-qa against the package. Depending on the software, manual testing may be required. Once satisfied, a changelog entry should be made, and the merge request should be assigned to a maintainer.

CNG

The pipeline will build a new set of images using the required software. An instance of the helm charts should be started, and testing done against that instance. Once complete, the MR should be assigned to a

maintainer for
merging.

charts/gitlab

The pipeline will install a review app version of the chart, and both spec and QA tests are run against the review app. Depending on the software, manual testing may be required. Once satisfied, a changelog entry should be made, and the merge request should be assigned to a maintainer.

Debug Renovate

To debug Renovate, check the logs and artifacts generated by the MR pipeline. An example can be found in this merge request.

Debug Renovate locally

It can be useful to debug Renovate locally for faster feedback, as the pipeline takes time to execute.

Here is a snippet that describes how to test locally.

Additional Information

Additional information can be found in Distribution: Renovate Gitlab Bot 101. Specific instructions for adding new dependencies are located in the Renovate Gitlab docs.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/delivery/_index.md

Common Links

Workflow	Team workflow
GitLab.com	@gitlab-org/delivery
Issue Tracker	Delivery
Slack Channels	gdelivery / @delivery-team
Delivery Handbook	Team training
Delivery Metrics	Metrics
Deployment and Release process	Deployments and Releases
Release Tools Project	Release tools
Release Manager Runbooks	release/docs/runbooks

Mission

The Delivery Group enables GitLab Engineering to deliver features in a safe, scalable and efficient fashion to GitLab.com, GitLab Dedicated and self-managed customers.

The group ensures that GitLab's monthly, and patch releases are deployed to GitLab.com, GitLab Dedicated and publicly released in a timely fashion.

Vision

The Delivery Group is a behind the scenes, primarily internal user facing team whose product and output has a direct impact on Infrastructure's primary goals of availability, reliability, performance, and scalability of all of GitLab's user-facing services as well as self-managed customers. The group creates the workflows, frameworks, architecture and automation for Engineering teams to see their work reach production effectively and efficiently.

The Delivery Group is focused on building a fully-automated deployment and release platform that builds on the CI/CD blueprint to enable fast, flexible releases and deployments with rapid rollout, failure detection and recovery.

Each member of the Delivery group is part of this vision:

- Each team member is able to work on all team projects
- The team is able to reach a conclusion independently all the time, consensus most of the time
- Career development paths are clear
- The team creates a database of knowledge through documentation, training sessions and outreach

Short-term

- Develop tooling and processes to allow the Maintenance Policy to be extended to fully support three versions
- Improve deployment pipeline observability to allow deployment inefficiencies to be measured and reduced
- Measure Release Manager workload to drive process improvements

Mid-term

- Significantly reduce Release Manager workload through improved processes and automation
- Develop flexible capabilities for routing traffic within our infrastructure to target desired resources for testing and deployments
- Develop the capability of creating new environments on-demand to aid with new testing requirements and rollout strategies
- Improve test execution flexibility to shift it left and allow simultaneous testing of multiple packages
- Scale security releases to meet organization needs whilst maintaining or reducing release manager workload
- Improve the tools and process we have for testing changes to the deployment and release tools to reduce the time it takes to develop and test tool changes

Long-term

- Support progressive deployments strategies to enable standard, experimental and risky changes

to be rolled out quickly, and safely with minimum impact on other deployment processes or MTTP

- Provide tooling to allow team members outside of Delivery to safely self-serve deployment and release tasks
- Fully automate the release processes used for Monthly, Security, and Patch releases for self-managed users to increase release velocity, and reduce Release Manager workload
- Significantly reduce MTTP below the current 12-hour target
- Increase the speed of deployment failure detection and recovery by enhancing health checks and rollback processes

Guiding Principles

These are a set of statements or questions we can use within the group to evaluate whether what we are doing is the right thing to do. When deciding on a piece of work to commit to, we should ask ourselves whether the work aligns with these principles. The principles will be driven by the Delivery Group Strategy, domain experts and Delivery Group Members. These are likely to change slightly over time as we learn and should not be considered static. Additionally, these principles are additive to the GitLab Values and we aim not to duplicate those here.

These principles are intended to help everyone work independently in a way that is aligned with our Group and strategy.

When designing a solution

1. Simplify Release Management. We always want to be reducing work for release managers either by removing tasks or by improving automation. Be very cautious about introducing new tasks without removing something else
1. Don't add functionality to Deployer. We aim to deprecate Deployer as part of our work to migrate away from VMs
1. Always consider metrics. If we don't have the metrics to track the value, we should look to add them
1. UX consistency. Strive for consistency across tool interactions and naming
1. Low-context designs. Design tools as if non-delivery team members need to use them. Keep things simple and let the tools do the hard work

When implementing a solution

1. Choose the smallest implementation. The less code we write, the less code we have to maintain (or debug)

Strategy

We've moved this subsection to the Delivery direction pages so that it's in the same place as the rest of our product direction.

Top-level Responsibilities

The group regularly works on the following tasks, in the order of priority:

1. Ensuring continuous delivery of GitLab application software to GitLab SaaS.(e.g. GitLab SaaS auto-deploy)

1. Coordination and preparation of GitLab releases for self-managed users for the monthly patch and security releases.
1. Participating in incident resolution and acting on corrective actions for SaaS and self-managed software delivery.
1. Increasing velocity of both SaaS and self-managed software delivery through foundational project work (e.g. Running GitLab SaaS on Kubernetes).
1. Improving the robustness of SaaS software delivery by creating and improving tooling (e.g. Deployment rollback).
1. Minimizing the use of custom tooling by building or enhancing features within GitLab (e.g. Create a Changelog feature).
1. Support other teams' needs related to software delivery on GitLab SaaS (e.g. New Container Registry deployment).

Teams

The Delivery Group is composed of two teams: Delivery:Releases and Delivery:Deployments.

The Delivery:Releases and Delivery:Deployments OKRs, while contributing to the wider GitLab objectives, are tailored and structured to achieve the Delivery Group Strategy as a single team.

Delivery:Releases

The primary goal of the Releases team is to provide everything to do with creating GitLab releases for customers and enabling internal customers to get their changes into releases. Major, minor and patch releases as well as a platform on which we can verify changes to various installation types (release environments) and create visibility for GitLab team members and customers into our release schedule (release dashboard).

Delivery:Deployments

The primary goal of the Deployments team is to provide everything to do with rolling out changes to active platforms managed by GitLab, including but not limited to continuous deployment to GitLab.com (deployment safety), changing gitlab.com architecture to support continuous deployment (eliminate staging ·) and optimizing the zero downtime rollout experience of GitLab Dedicated.

Team Members

The following people are members of the Delivery:Releases Team:

```
{{< team-by-departments departments="GitLab Delivery:Release BE,GitLab Delivery:Release SRE"
manager-role="Engineering Manager, Delivery:Releases" >}}
```

The following people are members of the Delivery:Deployments Team:

```
{{< team-by-departments departments="GitLab Delivery:Deploy BE,GitLab Delivery:Deploy SRE"
manager-role="Engineering Manager, Reliability Engineering" >}}
```

Team counterparts

The following members of other functional teams are our stable counterparts:

```
{{< engineering/stable-counterparts role="[,&] Delivery" >}}
```

Performance indicators

Delivery Group contributes to Engineering function performance indicators through [Infrastructure department performance indicators].

The group's main performance indicator is Mean Time To Production (MTTP), which serves to show how quickly a change introduced through a Merge Request is reaching production environment (GitLab.com).

At the moment of writing, the target for this PI is defined in this key result epic.

MTTP is further broken down into charts and tables at the Delivery Team Performance Indicators Sisense dashboard.

Delivery domain ownership between Delivery teams

The Delivery Group owns the tools and capabilities needed for GitLab deployments and releases. The diagram below shows the split of domain ownership between the two teams and the current release managers. Where the domain overlaps with teams outside of the Delivery Group, we focus primarily on the deployments and releases capabilities and needs.

- Diagram source

Release Manager ownership

Release Managers are members of the Delivery group but during their time as release managers, they're wearing a different hat. The primary customers are GitLab users.

1. Auto-deploys: Release Managers operate the auto-deploy process. Largely this will make use of capabilities provided by the Deployments team, but the Orchestration tools will be making use of the Deployments team capabilities. Environment health checks are an example of a Deployments capability that will be integral to the process and tools the release managers use.
2. Self-managed releases: Release Managers operate the release processes (patch and security) using the capabilities provided by the Releases team.
3. Post-deploy migrations: Release Managers operate the PDM process using the capabilities provided by the Deployments team.
4. Hot patch process: Release Managers, working with EOCs, will manage the hot patch process. Hot patch capabilities are provided by the Releases team with heavy dependence on Deployments capabilities due to the shortened process and therefore reduced pipeline jobs.
5. Deployment blockers: Release Managers are responsible for identifying, and reporting on deployment blockers in order to provide the teams with data needed to plan improvements.
6. Release Manager dashboards: Release Managers own <<https://dashboards.gitlab.net/d/delivery-releasemanagement/delivery-release-management?orgId=1>> plus have the freedom to create any additional dashboards that they think would be useful for release management. The data needed for dashboards will be made available from a centralized place, owned by Deployments.
7. Escalation of Deployment Blockers. When facing deployment blockers in any environment of

more than 2 hours without a clear path to resolution, escalate to the Release Management Escalation Schedule in PagerDuty to help coordinate and unblock deployments.

Delivery:Releases ownership

The primary customers of the Releases team are:

- the internal GitLab users who want to deploy & release changes i.e.,
- Release Managers and Stage Groups.
- customers for the monthly Release

1. Release metrics: Providing A centralized store for metrics related to releases. Deployments will primarily be concerned with providing a metrics capability to allow all deployment pipelines to record metrics in a useful way to fuel all required dashboards.

1. Releases/Packages Pipeline visibility: Providing visibility of pipeline configuration, status, and outcome.

1. Release change management tooling: Providing the ability for changes to be included, or excluded from releases to Self-Managed users.

1. Release/packaging execution log: Ensuring that an accurate log of releases is maintained.

1. Deployment & release metadata: Tracking component versions and dependencies to allow for quality gates to be accurate, and to ensure predictable releases.

1. QA test execution & results visibility: Ensuring that all deployments and releases pass the required testing. Releases will be particularly concerned with timing of test execution and making sure that the correct dependencies are in place for reliable results.

1. Releases dashboards: Delivery:Releases will own a set of dashboards to guide the team's work on designing effective release processes. Dashboards, or templates, will also be needed to evaluate the effectiveness of individual release pipelines.

1. Release Publishing: Publishing packages to various distribution sites (e.g., packages.gitlab.com, Docker Hub, etc.), publishing tooling, and guaranteeing a reliable publishing process.

Delivery:Deployments ownership

The primary customers of the Deployments team are:

- Release managers who rely on the deployment tooling
- GitLab SaaS (GitLab.com, Dedicated and Cells) customers expecting deployments of updated code to their various infrastructures.

1. Deployment changelock: will make sure that all deployments observe planned and ad-hoc changelocks. Examples include PCLs, S1/S2 incidents, as well as other Change Requests.

1. Environment changelock: will make sure that environments can be locked to schedule, or on an ad-hoc basis if required by planned maintenance or poor environment health. Guaranteeing that changes are rolled out in a predictable way will also be a Deployments responsibility.

1. Environment health: Ensuring that environment health is assessed and available to guide deployment decisions.

1. Release & deployment metrics: Providing A centralized store for metrics related to deployments & releases. Deployments will primarily be concerned with providing a metrics capability to allow all deployment and release pipelines to record metrics in a useful way to fuel all required dashboards.

1. Deployment change management tooling: Providing the ability for changes to be included, or excluded from deployments.
1. Deployment execution log: Ensuring that an accurate log of deployments is maintained.
1. Application rollout: will be responsible for applying changes to the required clusters and environments. Rollout strategies, e.g., canary, blue/green, with gradual traffic increase etc, will be capabilities provided by Deployments.
1. Rollout dashboards: will own a set of dashboards to guide the team's work on managing effective rollouts to all environments. Examples could include the timing of changes applied to individual servers and visibility into environment use.
1. Canary environments: will own the rollout capability of the canary environments. They'll work closely with Reliability to ensure full environment management.
1. Deployment and release test environments: Pre, Staging, Release: will own the rollout capability of the test environments. They'll work closely with Reliability to ensure full environment management.
1. Deployment and release production environment: will own the rollout capability of the Production environment. They'll work closely with Reliability to ensure full environment management.
1. Deployment dashboards: Delivery:Deployments will own a set of dashboards to guide the team's work on designing effective deployment processes. Dashboards, or templates, will also be needed to evaluate the effectiveness of individual deployment and release pipelines.

How we work

Reaching our Team

Reason	Contact	Via
S1/P1 Deployment/Release related issues	@release-managers	Slack
Any Priority Deployment/Release related issues	@gitlab-org/delivery	GitLab

Release Managers have a weekday follow-the-sun rotation and can be reached via the @release-managers handle on Slack. For weekend support or other escalations please use the Release Management Escalation steps below to reach Delivery Leaders.

Release Management Escalation

During weekday working hours you can reach the current Release Manager via the @release-managers handle on Slack.

For release management support outside of working hours, or if you need to escalate to Delivery Leadership please follow the steps below to page using PagerDuty.

1. Open the PagerDuty app in Slack and use the /pd trigger slack command to trigger a new incident
2. Select the Release Management Escalation service and provide request details in the Title field. You can leave all other fields empty
3. Click Create and one of the Delivery leaders will respond

Project Management

The Delivery Group's work is tracked through a number of epics, issues, and issue boards.

Epics and issue boards are complementary to each other, and we always strive to have a 1-1 mapping between a working epic and an issue board.

Epics describe the work and allows for general discussions, while the issue board is there to describe order of progress in any given epic.

Each project should have a project-label applied to all epics and issues to allow issue boards to show a full project view.

Issues should primarily be created in the delivery issue tracker for visibility and prioritization. Repositories that Delivery maintain should have their own issue trackers disabled. The purpose of this is to ensure a single source of truth for work to be prioritized and visible within the team.

Epics

The Release Velocity epic tracks all work related to the group mission. Any working epic that the team creates should be directly added as a child to this top level tracking.

Working epic should always have:

1. A Problem Statement.

1. Directly responsible individuals responsible for the project completion.

1. Defined exit criteria

1. Issue admin section to provide the issue priority, labels and epic for quick actions.

Example.

1. Status YYYY-MM-DD to indicate what is being worked on, why, and planned next steps. The DRI is responsible for updating the epic status every Wednesday. Note, this must be the last heading in the epic to support automated epic summary updates

1. Start date and estimated due date

1. Labels:

-

- Label used as part of the project scope (eg. kubernetes, security-release). The DRI should create a suitable project scope label and add it to the Delivery-triage rules if needed.

- Epic status label using the 'workflow-infra::triage', 'workflow-infra::proposal', 'workflow-infra::in-progress', 'workflow-infra::done'

In cases where the work is tracked in a project in a different group outside of our canonical project location, we will create two epics for the same topic and state in the epic description which one is the working epic.

Issue Boards

Each working epic should be accompanied by an issue board. Issue boards should be tailored to the specific project needs, but at minimum it should contain the workflow labels shown on the workflow diagram.

Labels

The canonical issue tracker for the Delivery group is at `gl-infra/delivery`. Issues are automatically labeled if no labels are applied using the triage ops project. The default labels defined in the labeling library.

By default, an issue needs to have a:

- 1. Workflow Label - Default: `workflow-infra::Triage`
- 1. Priority Label - Default: `Delivery::P4`
- 1. Group Label - `group::delivery`
- 1. Team Label - `team::Delivery-Deployments` or `team::Delivery-Releases`
- 1. Other Label - project or team management related label.

Workflow

The Delivery group leverages scoped `workflow-infra` labels to track different stages of work.

Not every issue will be prioritised for building as soon as it is ready. Instead we manage a Build board with all `workflow-infra::In Progress`, and `workflow-infra::Ready` issues focused on the team's current goals.

The standard progression of workflow is described below:

```
mermaid
sequenceDiagram
    participant ready as workflow-infra::Ready
    participant progress as workflow-infra::In Progress
    participant done as workflow-infra::Done
    ready ->> progress: 1
    Note right of ready: Issue is assigned and work has started.
    progress ->> done: MR is merged and deployed to production
    Note right of progress: Issue is updated with rollout details, workflow-infra::Done label is applied, issue can be closed.
```

There are three other workflow labels of importance omitted from the diagram above:

- 1. `workflow-infra::Cancelled`:

- Work in the issue is being abandoned due to external factors or decision to not resolve the issue. After applying this label, issue will be closed.

- 1. `workflow-infra::Stalled`

- Work is not abandoned but other work has higher priority. After applying this label, team Engineering Manager is mentioned in the issue to either change the priority or find more help.

- 1. `workflow-infra::Blocked`

- Work is blocked due external dependencies or other external factors. After applying this label, issue will be regularly triaged by the team until the label can be removed.

Label workflow-infra::Done is applied to signify completion of work, but its sole purpose is to ensure that issues are closed when the work is completed, ensuring issue hygiene.

Priority Labels

The Delivery group uses priority labels to indicate order under which work is next to be picked up. Meaning attached to priorities can be seen below:

| Priority level | Definition |

| ----- | ----- |

| Delivery::P1 | Issue is blocking other team-members, or blocking other work. Needs to be addressed immediately, even if it means postponing current work. |

| Delivery::P2 | Issue has a large impact, contributes towards current OKRs or will create additional work. Work should start as soon as possible after completing ongoing task. |

| Delivery::P3 | Issue should be completed once other urgent work is done. |

| Delivery::P4 | Default priority. A nice-to-have improvement, non-blocking technical debt, or a discussion issue. Issue might be completed in future or work completely abandoned. |

The group uses priority labels differently to the general issue triage priority definition in order to avoid ambiguity that comes with difference in timelines between Stage teams and Infrastructure teams. We have different timelines (release brings different expectations for Delivery), different DRI's (no PM for Delivery), and different importance (Blocked release means that no one can ship anything).

Other Labels

Some of the labels related to the team management are defined as:

1. onboarding - issues are related to granting access to team resources.
1. team-tasks - issues related to general team topics.
1. Discussion - meta issues that are likely to be promoted to a working epic or generate separate implementation issues.
1. Announcements - issues used to announce important changes to wider audience.

Project labels are defined as needed, but they are required unless the issue describes a team management task.

Incidents impacting Delivery may optionally include an impact label.

Delivery:Deployments specific ways of working

In addition to the Epics, Issue Boards, Labels, and Workflow practices common to the entire team, the Delivery:Deployments team adopts a few extra approaches to improve its communication and better share knowledge and decisions within the team.

- Each Epic should contain a Decision Log: this helps to keep a single, centralized SSOT where all decisions taken during project work are listed. For completeness, we should report the date

of the decision,
who was involved in the discussion and decision, and the outcome of the decision.

For simplicity, you can use the snippet below to add Decision Log to Epics Description:

markdown

Decision Log

<details>

<summary>Decision List</summary>

<details>

<summary>YYYY-MM-DD</summary>

<Discussion details, people involved in the discussion and decision, decision outcome>

</details>

</details>

- Each Issue should contain a Progress Thread as a comment: this helps make the work visible in an async environment and shares knowledge while the work progresses.

Comments within the progress thread should highlight the progress achieved, the intermediate steps/results we got, assumptions, discoveries, and blockers we face.

This approach allows people from the team and outside of the team to build a clear idea and eventually contribute with comments and suggestions.

Choosing something to work on

The Delivery group generally has working epics assigned to a DRI who is responsible for making sure work is broken down into issues, and appropriate issues are moved onto the Build board to keep the project on track. However, anyone is welcome to pick up any tasks from the Build board regardless of which project it belongs to.

Merge Requests

The Delivery group respects the Company principle of everything starting with a merge request.

1. All Merge Requests (MRs) must go through the review process.

1. MRs should be labeled with the Delivery labels.

1. It is expected that MR author assigns reviewers once the MR is ready to go.

1. Follow GitLab Code Review Guidelines.

1. We dogfood the Reviewers feature to assign reviewers to an MR.

Besides, we try to apply some best practices when doing Merge Requests:

- Choosing reviewers: Commonly, any team member on Delivery can be assigned for reviews with a few considerations:

- We avoid adding the entire Delivery group to Merge Requests Review to avoid too many pings.

- GitLab team is globally distributed, so think about timezone when assigning someone as reviewer.

- Project - The reviewer should be knowledgeable or at least familiarized with the project,

for example, release-tools reviews are normally handled by backend engineers, while k8s-workloads reviews are handled by SREs.

- Context - If you're working closely with a peer, it's recommended to assign it to this team member for shorter review cycles.
- Release manager (or capacity) - If a team member is a release manager and they're working on a release task, they should not be bothered with reviews.
- The usual code-review turnaround is two business days.
 - This doesn't apply if the merge request is associated with a ~Delivery::P1 item. In that case, the Merge Request review has to be treated with priority and urgency.
 - If the merge request has all the required approvals it can be merged by the author.
 - If you are assigned as a Reviewer, and you think to not be the best fit for the Merge Request (out of working hours, not enough understanding, don't have the capacity to review, etc.), please mention this to the merge request author so they can find another maintainer available or select an alternate team member that you feel would be better suited.
- For Delivery team projects, one approval is usually sufficient to merge an MR, but the reviewer or author can request another review if they prefer. A second review might be requested because the reviewer is not comfortable being the only reviewer, or the author would like a second pair of eyes, or for any other reason.

Project demos

As part of the project, we might decide to organize project demos. The decision on creating a demo depends on the expected longevity of the project, but also on the complexity of it.

The purpose of the demo is to ensure that everyone who participates in the project has a way of sharing their findings and challenges they might have encountered outside of the regular async workflow. The demos do not have presentations attached to them, and they require no prior preparation.

The demoer shouldn't feel like they have to excuse themselves for being unprepared, and expect that their explanation without faults. In fact, if what is being demoed is showing off no weaknesses, we might have not cut scope in time.

It is encouraged to show and discuss:

1. The imperfectness of the specific code implementation.
1. How broken the used tool might be.
1. How an established process breaks down.
1. How challenging a problem being resolved might be.

Team training

Every Delivery Group member is responsible for sharing skills either through creating a training session for the rest of the group or through paired work. See the page on team training for details.

History

The Delivery team officially came into existence on 2018-10-23. This was the culmination of a larger alignment that was happening throughout that year, exposed by the need to streamline releases for self-managed users and creating a better experience for GitLab.com users.

All throughout GitLab's existence, Release Management had been a monthly rotating role served by developers. The idea behind it was to keep developers close to the whole lifecycle of the software they create, and ensure that they automate their work. This worked well until the number of application changes, and developer tasks grew too large for anyone to handle as a secondary task. The event that indicated the need for a change was a near miss event near the end of 2017, when the first Release Candidate was deployed to GitLab.com just 2 days before the 22nd. That whole month was riddled with challenges, from release managers struggling to deliver their day to day development tasks and RM tasks, to multiple unsuccessful deployments to GitLab.com. Most importantly, this was a first indication that the company was growing and that the processes that worked previously, might need to change to accommodate the larger growth that was planned.

After some internal discussions, we entered 2018 with an attempt to work on process improvements, rather than changing everything in one go. We went from a monthly rotation to two month release manager rotation, started noting down spent time. Over the next several months we'd seen general stabilization of the process but it became apparent that spending 4 engineers time in Release Manager rotation was not getting us anywhere closer to improving the deployment process for GitLab.com, and with each developer we hired the task list grew bigger.

The initial discussion on what is in front of us to achieve Continuous Delivery on GitLab.com exposed a clear need for a team focused on this specific task.

With the team created, we set out to work on the release we designed, change the way we deploy to GitLab.com, merge GitLab Rails codebases and many more other tasks.

After a successful team onsite (aka Fast boot) where we executed on our tasks while being in the same room together for the first time, we announced the first step towards Continuous Delivery on GitLab.com. This was a very large change that changed the deployment frequency from deploying from the default branch once per month (for the total of 4-6 deploys to include bug fixes), to taking commits from the default branch once per week.

The team focus then shifted to getting deployment time measured in hours, and migration of GitLab.com to Kubernetes.

Prior to 2020, the team impact overview was created in Slack, and in the years that followed the overview was logged in issues:

- 1. Year overview for 2020
- 1. Year overview for 2021
- 1. Year overview for 2022
- 1. Year overview for 2023
- 1. Year overview for 2024

SECTION: engineering/infrastructure-platforms/gitlab-delivery/delivery/engineering-manager.md

Moved Readme of Marin Jankovski to the new location.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/delivery/metrics.md

Metrics

Delivery team uses Mean Time To Production (MTTP) as an indicator of our speed capabilities to deploy application changes into production.

- Performance Indicator
- Target: 12 hours.

```
{{< tableau height="900px" toolbar="hidden"
src="https://10az.online.tableau.com/t/gitlab/views/InfrastructureKPIs/MeanTimetoProductionKPI"
>}}
{{< /tableau >}}
```

MTTP measures the elapsed time (in hours) from merging a change in gitlab-org/gitlab project master branch, to deploying that change to GitLab.com.

MTTP breakdown

MTTP is conformed by:

- Time to inclusion: The merge request is merged and waiting to be included in an auto-deploy branch.
- Time to package: The package to be deployed is being built.
- Time to staging: Time it takes for the package to be deployed to staging and tested.
- Time to canary: Time it takes for the package to be deployed to canary and tested.
- Time to production: Time it takes for the package to be deployed to production.

To measure the MTTP subcomponents, the following metrics are used:

- Deployment SLO

These allows to decide whether we need to focus on speed or safety would have the biggest impact on further reducing MTTP.

Deployment SLO

- Target: 8 hours.
- Dashboard

Deployment SLO measures the deployment frequency and duration by tracking the percentage of deployment pipelines that complete within the target duration. Deployment duration is measured as the elapsed time between a starting on staging canary (gstg-cny) through the completion of the deployment on Production (gprd). Visit the Releases page for an

end-to-end overview of the various environments and deployment process.

Next: Include Deployment SLO metric as part of our release process

Release Manager workload metrics

The Release Manager workload metrics - internal link track the time needed for Release Management tasks.

Each process is broken down into key parts to help identify where the majority of the effort is spent. Overall times are tracked and an average allows changes over time to be observed.

More details about what is being measured, and how, can be found in the release docs.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/delivery/training.md

Common links

Delivery Team Handbook

Purpose

The purpose of the team training is to share the work done with the rest of the team, company and public.

It also allows the rest of the team to easily transition into new features and projects and prepare them for maintenance. Training is public and available for everyone wanting to get familiar with the topics.

Training should either be a dedicated training session, or pairing on a task that is currently being worked on. In both cases the session should be recorded and shared on our Youtube playlist, or at minimum learnings should be used to improve and expand our documentation.

Training should be:

- Providing a high level overview of the invested work
- Describing the main challenges encountered during development
- Explaining the possible pit-falls and specificities

Training should not be:

- Replacement for documentation
- Replacement for writing down progress in issues
- Replacement for follow up issues

Simply put, the training is a summation of: notes taken in issues during development, programming challenges, high level overview of written documentation. Your team

member should be able to take over the maintenance or build on top of your feature with less effort after they have been a part of the training.

Note Do not shy away from being technical in your training. You can ask yourself: What would have been useful for me when I started working on this task? What would have helped me be more efficient?

Efficiency of the training

In order to see whether the training is efficient, Delivery lead will rotate team members on projects where training was done. For example, if the feature requires regular releases, the person who gave the training will be considered a tutor. Different team member will follow the training and documentation and will ask the original maintainer for help. The new person responsible is now also responsible for improving the feature. They are now also responsible of training other team members.

Release Manager discussions and fire drills

Delivery team members organize monthly practice runs to share knowledge and skills about release management topics. These sessions are focused on release processes, fire drills and any other topic that could ramp-up the release manager knowledge between Delivery team members.

The topics and scenarios discussed in the meeting are brought up in Release Manager agenda (internal only) ahead of the meeting time and the videos are posted on the [Release Manager discussions and fire drills playlist].

Training Listings

Youtube playlist

Release Manager discussions and fire drills playlist

FAQ

Q: Isn't this double work?

A: No. The training should be prepared while documenting the task or shortly after.

Q: Won't this slow me down?

A: At the beginning, possibly. However, every hour of the training given will multiple the value of it by the amount of team members and people involved related projects.

Q: Isn't it more useful to let the team check out the docs and ask questions?

A: In an ideal world, possibly. However, everyone has a lot of tasks assigned and they might not be able to go through the docs until they need to do something. This might be months later and you, as a person who would give the training, might not be able help efficiently anymore.

SECTION: engineering/infrastructure-platforms/gitlab-delivery/distribution/_index.md

Overview

The Distribution team is composed of two subgroups: Distribution:Build and Distribution:Deploy

Build team is focused on producing artifacts including system packages, container images, and related components like marketplace listings along with tooling required to create and maintain them.

Deploy team is focused on installation and upgrade mechanisms to ensure smooth deployments. This includes system integration, scripting, templating, and related configuration management tooling.

In addition to product deliverables, both groups review a large number of MR's authored outside the team. These include dependency and security updates along with configuration controls and other bundled components like PostgreSQL, Consul, Patroni.

Target Consumers

The primary user persona for Distribution are system administrators who are responsible for managing GitLab instances. Team goals are to make it as easy as possible to deploy, upgrade and configure GitLab on a range of on-prem and cloud platforms at a variety of scales.

Deployments include everything from single node deployments for evaluating GitLab all the way through the 50K user reference architecture and beyond. The primary goal is to ensure end users have a high-speed, low-friction experience when managing GitLab with limited downtime or service disruptions.

Omnibus packages, Helm Charts and Operators are primary deployment methods Distribution currently supports.

Distribution Build

<!-- markdownlint-disable MD051 -->

Team |
Charter

Responsibilities:

- Research for the support of new clouds, platforms, architecture, and components
- Access controls, permissions, and CVE patches
- Dependency updates
- License management
- Submissions to Partners for validations/certifications
- The install, update, and upgrade pages
- Build and own the infrastructure used for creating the various installation methods
- Maintaining infrastructure used in Distribution

Distribution Deploy

Team

Charter

<!-- markdownlint-enable MD051 -->

Responsibilities:

- Initial installation and composability for self-managed installations and GitLab.com
- Upgrades / Downgrades
- Scaling deployments
- Migration between platforms or providers
- Data lifecycle management
- Secure configuration & communication
- Research of clouds and platforms for integration to existing tools

Vision

Distribution team vision is shown below, and not limited to:

Technology

- GitLab has an official installation method on all major platforms and architectures.
- GitLab offers official one click installation method on all major cloud platforms.
- GitLab is able to automatically upgrade itself safely and reliably.
- Official repositories for any Kubernetes package repository.
- GitLab runs equally well on both high and low resource systems (such as Raspberry Pi).
- GitLab.com is running using the official installation methods.
- All GitLab features are installed and configured by default.
- Installation interface to set up and configure GitLab.
- Any GitLab installation is able to report installation/upgrade errors automatically.
- Setting up GitLab in HA configuration is automated and simple.
- All installation methods are automatically tested before release.
- Most frequently used configuration options are tested using end-to-end integration tests.

Team

- Each team member is able to work on all team projects along with the ability to focus on specific technologies.
- Each team member is a part of a hiring panel aimed to hire better than the best in the team.
- Team creates documentation to increase knowledge and awareness to support a self-service model.
- Team is able to reach a conclusion independently all the time, consensus most of the time.
- Team has official certifications for frequently used technologies and platforms.
- On-boarding and off-boarding is efficient.
- Career development paths are clear.

Mission

Distribution ensures the experience of installing and maintaining GitLab is easy and safe for everyone. It is Distribution team's task to think about the widest variety of installation/update use-cases and provide a solution that will satisfy most needs. Distribution

is there to provide the best possible experience for a user that is a novice but also a veteran when it comes to installing and maintaining software.

Distribution:Build Charter

Build team focus is ensuring GitLab components are tested, current, license compliant, and available for our users' platforms and architectures. This group manages the build pipelines, researches support for new services, platforms, and architectures, as well as maintains existing ones. We strive to respond efficiently to build failures, security results, and dependency changes in order to ensure a safe reliable product for our users.

Distribution:Deploy Charter

Deploy team focus is configuration, deployment, and operation of GitLab as a whole product. The goal is to deliver an intuitive, clear, and frictionless installation experience, followed by smooth, seamless upgrade and maintenance processes for deployments of any scale. We strive to deliver ongoing operational behaviors for scaling, little to zero downtime upgrades, and highly reliable experiences for not only instance administrators but their users.

Goals

Increase of active installations

Reduce average days behind latest version

Team members

Distribution Build Team

The following people are members of the Distribution:Build Team:

```
{{< team-by-manager-slug manager="denisra" team="Build" >}}
```

Distribution Deploy Team

The following people are members of the Distribution:Deploy Team:

```
{{< team-by-manager-slug manager="plu8" team="Deploy" >}}
```

Stable counterparts

The following members of other functional teams are our stable counterparts:

```
{{< engineering/stable-counterparts manager-role="Backend Engineering Manager, Distribution" role="Core Platform:Distribution" >}}
```

Common links

- Distribution team issue tracker
- Slack chat channel

Team responsibility

In addition to the separate responsibilities for Build and Deploy. The Distribution team as a whole is responsible for:

1. The omnibus-gitlab installation package.
 - Installation using package is simple, quick, secure and protects the data integrity.
1. A cloud native installation method.
 - Installation using the Helm charts is able to scale easily with the increased demand.
1. Triaging issues and Reviewing Merge Requests in all owned projects.
1. Providing regular demonstrations of recent solutions or works in progress

Team objectives

Based on team responsibilities, the following objectives apply:

- Distribution:Build
 - Installation pages need to be complete, correct, easy, helpful, and attractive. Even if team's primary skill is not design and UX, team can maintain the pages with the help of teams that have this skill as their primary.
 - Nightly builds are installed on dev.gitlab.org using one of the official artifacts of the Distribution Build team. Purpose of this exercise is to have a production level instance using one of the installation methods that the community will be using.
- Distribution:Deploy
 - The omnibus-gitlab installation package and the cloud native installation method should allow a beginner to install GitLab quickly, correctly and completely. Required configuration should be minimal, and it is the Distribution Deploy team's responsibility to automate as much of this configuration as possible for the user.
 - The omnibus-gitlab installation package and the cloud native installation method should be sufficiently configurable to allow an advanced user a way of setting up a more complex GitLab architecture.
 - The above two goals might sound opposing, however they are not. A compromise can be made between these two goals without increasing the project maintenance cost for the Distribution team. We optimize for the best initial installation experience and then expand to further complexity.
- Distribution Common Objectives
 - Triaging issue tracker is a task that allows us to keep on the pulse of the changes we make. By being in contact with users and customers, we can maintain visibility into most frequently reported bugs or requested features. Not everything reported will be resolved, however all of the reports should be triaged. This also applies to mentions in GitLab CE/EE repositories on issues with Distribution and group::distribution label.
 - Every Distribution team member is responsible for creating a training session for the rest of the team. See the page on team training for details.
 - When the team that manages GitLab.com creates an issue, the item should be raised up directly to the team Engineering Manager and Product Manager. While these issues are important, we don't necessarily need to provide a complete solution right away, but we need to work with the other team on providing a path forward with their request.

Primary Projects

omnibus-gitlab - This project creates platform-specific, self-contained GitLab packages and images for self-managed consumption in cloud environments and on-premises hosting.

Cloud Native GitLab provides cloud native containers to deploy GitLab. These containers may be deployed and managed via Helm using GitLab Charts or GitLab Operator on Kubernetes, OpenShift, and Kubernetes compatible container platforms.

Omnibus GitLab project's product outputs

```
mermaid
graph TD
  subgraph Code
    OG --> PKG
    PKG --> DOI
    PKG --> AMI
  end

  subgraph Deploy
    DOI --> DOK
    DOK --> DOC
    AMI --> AWS
    PKG -.-> GET
    GET --> GCP
    GET --> AZURE
    GET -.-> |Future|AWS
    GET -.-> |Future|VMW
  end

  OG[Omnibus GitLab]
  PKG[Linux Package]
  DOI[Container Image]
  DOK(Docker)
  DOC(Docker Compose)
  AMI[Amazon Machine Image]
  GET[GitLab Environment Toolkit]
  AWS(AWS)
  GCP(Google Cloud Platform)
  VMW(VMWare)
  AZURE(Azure)
```

Cloud Native GitLab project's product outputs

```
mermaid
graph TD
  subgraph Code
    CNG --> HC
    CNG --> GOP
    HC --> GOP
  end
end
```

```
subgraph Deploy
  GOP --> K8s
  GOP --> OS
  CNG --> DC
  HC --> K8s
end
```

CNG[Cloud Native GitLab containers]
HC[Helm Chart]
K8s(Kubernetes)
GOP[GitLab Operator]
OS(OpenShift)
DC(Docker Compose)

All Projects

Name	Location	Description
Omnibus GitLab	gitlab-org/omnibus-gitlab	Build Omnibus packages with HA support for LTS versions of all major Linux operating systems such as Ubuntu, Debian, CentOS/RHEL, OpenSUSE, SLES
Docker All in one GitLab image	gitlab-org/omnibus-gitlab/docker	Build Docker images for GitLab CE/EE based on the omnibus-gitlab package
GitLab Helm Chart	gitlab-org/charts/gitlab	Cloud Native GitLab Helm Charts
Docker images for GitLab Helm Chart	gitlab-org/build/CNG	Individual images used by GitLab Helm Charts
GitLab Operator	gitlab-org/cloud-native/gitlab-operator	The GitLab operator creates and manages GitLab instances/deployments in a container platform such as Openshift or Kubernetes. It will run in any environment that provides native Kubernetes resources including deployments, statefulsets, services, ingress/openshift routes, persistent volume claims, persistent volumes, etc.
Kubernetes Helm Charts index	charts/gitlab.io	Helm charts repository index
AWS images	AWS marketplace	AWS image based on the omnibus-gitlab package
Reference Architecture Tester	gitlab-org/distribution/reference-architecture-tester	Spins up reference architecture based GitLab deployments using GET and runs QA against them
Omnibus GitLab Builder	GitLab Omnibus Builder	Create environment containing build dependencies for the omnibus-gitlab package
Licenses of bundled dependencies	Licenses page on GL Pages	Webpage listing the bundled dependencies in each package along with their license.

Working with the community

The install and upgrade process is one of the first features that systems administrators experience when working with GitLab.

As a result, the projects managed by the Distribution team have a high level of engagement by the user-base. The GitLab community is made up of more than just code contributors; users logging issues and feature requests are constantly pushing us forward and helping create a better experience.

In Distribution we strive for the following in our public projects:

1. Uphold our Community Code of Conduct.
1. Enable GitLab's mission that everyone can contribute..
1. Show our work in public.
1. Recognize and thank contributors for their work.
1. Respect contributors donated time by providing a timely review turnaround time.

Working with Open Source communities

The open core of GitLab is built on top of thousands of open source dependencies. These dependencies and their communities are important to the GitLab strategy, and working with these dependencies is an essential part of the projects the Distribution team maintains.

In Distribution we strive to:

1. Consider the impact of our work on the open source communities that we benefit from.
1. Promote the importance of these open source communities within GitLab.
1. Raise issue with any decision that goes against our stewardship promises.
1. Find opportunities to contribute back the changes we make.

Public by default

All work carried out by the Distribution team is public. Some exceptions apply:

- Work has possible security implications - If during the course of work security concerns are no longer valid, it is expected for this work to become public.
- Work is done with a third party - Only when a third party requests that the work is not public.
- Work has financial implications - Unless financial details can be omitted from the work.
- Work has legal implications - Unless legal details can be omitted from the work.

Some of the team work is carried out on our development server at dev.gitlab.org. Infrastructure overview document lists the reasons.

Unless your work is related to the security, all other work is carried out in projects on GitLab.com.

If you need to submit a sensitive issue, please use confidential issues.

If you are unsure whether something needs to remain private, check with the team Engineering Manager.

YouTube Playlists

The team regularly publishes demos, discussions and meetings to these playlists:

- Distribution Team Demos (Public) More on team demos.
- Distribution Team Discussions (Mostly public, but some private content)
- Distribution Team Meetings (Private)

Onboarding and offboarding

In addition to general company on-boarding and off-boarding, Distribution team has its own process to get new team members up to speed more quickly.

If you are starting with your onboarding, open an issue in Distribution team issue tracker, select Team-onboarding template and assign the issue to yourself.

Going through the steps noted in the issue should be your top priority, higher than the general company on-boarding issue. This is because items in team on-boarding are specific to your role and it will allow you to get up-to-speed quicker.

Off-boarding should be carried out by the Engineering Manager of the team, using the appropriate issue template in the same issue tracker.

Work Resources

General resources available to developers are listed in the [Sandbox cloud page](#).

In the Distribution team specifically, everyone should have access to the following resources:

- Google projects in Google Cloud Platform
 - testground
 - cloud-native
 - omnibus-build-runners
- AWS build infrastructure
 - Distribution group AWS sandbox account
 - cloud-native EKS cluster for CI (requires a maintainer to grant access)
 - GitLabTop account (To be retired, existing team members only)

If you don't have access to any of these resources, create an Access Request and assign it to your manager for approval.

Infrastructure and maintenance

As part of the team responsibilities, team owns maintenance of infrastructure used for day to day work.

For list of nodes and description of the maintenance tasks, see the [infrastructure and maintenance page](#).

Team workflows

General engineering workflow applies to the Distribution team. Since Distribution is working across multiple projects, our team workflow is further explained and summarized on the [Distribution workflow page](#).

Further reading

The following important areas of the GitLab Handbook impact how we work and are worth reading.

- Distribution workflow page
- General engineering workflow page
- How to reinforce our values
- Continue to serve smaller users (internal only)
- Promises to our Open Source community
- How to follow our Product Principles
- Principles of our company strategy
- Effective & Responsible Communication Guidelines
- Test Platform in Distribution group

Work/life harmony

Working all-remote and asynchronous first

offer flexibility in how team members approach their workday. Team members must make choices on how best to balance work time with other areas of life.

For new team members, the following resources provide examples on how to focus their time:

- How team members approach their day
- Blog post: A day in life of a remote worker
- The option of a non-linear workday
- GitLab handbook: Work/life Harmony

The following GitLab Handbook areas are key in maintaining a healthy work/life balance.

- Family and Friends First, work second
- Combating burnout, isolation, and anxiety in the remote workplace
- Recognizing Burnout

How to work with Distribution

Everything that is done in GitLab will end up in the supported installation methods that are distributed to users. While that sounds like the last link in the chain, it is one of the most important ones. This means that informing the Distribution team of a change in an early stage is crucial for releasing your feature. While last minute changes are inevitable and can happen, we should strive to avoid them.

We expect every team to reach out to the Distribution team before scheduling a feature in an upcoming release in the following cases:

- Reach out to Distribution Build when:
 - The change requires a new or an update on a gem with native extensions.
 - The change requires a new or updated external software dependency.
 - Also when the external dependency has its own external dependencies.
- Reach out to Distribution Deploy when:

- The change adds, modifies, or removes files that should be managed by omnibus-gitlab. For example:

- The change introduces new directories in the package.
- The change introduces new files in previously not defined locations
- The change requires a new configuration file.
- The change requires a change in a previously established configuration.

To sum up the above list:

If you need to do install, update, make, mkdir, mv, cp, chown, chmod, do compilation or configuration change in any part of GitLab stack, you need to reach out to the Distribution team for opinion as early as possible.

This will allow us to schedule appropriately the changes (if any) we have to make to the packages.

If a change is reported late in the release cycle or not reported at all, your feature/change might not be shipped within the release.

If you have any doubt whether your change will have an impact on the Distribution team, don't hesitate to ping us in your issue and we'll gladly help.

Requesting a new component or replacing a component

Create a Deliverables Request
in the Distribution team [issue tracker].

Requesting feedback on a feature change

For requesting feedback you can ping @gitlab-org/distribution in your issue or merge request.

When looking for a Distribution reviewer for a merge request you should use the process outlined in our merge request workflow for projects owned by the Distribution team. For changes that need Distribution review outside those projects, please ping @gitlab-org/distribution in the merge request.

Engaging Distribution for expertise in support

GitLab provides a unified process to open a request for help (RFP) to support customers. This process is in place to ensure we have a single source of truth for those, so that we better collaborate cross-functionally as, many times, the requests actually require expertise of multiple areas of the product, or is not initially clear which area is more suitable to support the customer. When sharing the information with multiple relevant groups, within the same support request process, we're able to get to a solution much more efficiently.

To open an RFP, refer to the procedures of our how to get help

handbook page.

This process allows us